



iQ-Studio 課程講義

黃誌祥 | 智能周邊應用事業處 軟體研發部 副理 · 2026-07-30

AGENDA

- 實作里程碑
- iQ-Studio 定位
- 實作流程說明
- 動手做

iQ9 實作演練 · 上午場 · iQ-Studio

規格看再多，不如親手讓它跑一次。

從零，把一台 IQ9 跑起 AI。

平台概覽 → 韌體燒錄 → 環境初始化 → 跑起官方 demo。 **全程動手**

Flash → Init → Run

三個里程碑，全程動手

里程碑

1



燒錄 BSP



2



環境初始化



3



跑起 demo



動手實作



韌體燒錄

把 IQ9 燒成可開發的 OS (PCAT / QDL) 。



./install.sh

主機用 ADB 連板；板子裝好 iQ-Studio 。



iqs-launcher

一行 iqs-launcher 起 YOLOv10n / Streampipe / VLM 。

Platform · 你今天在哪台機器上

QCS9075 (Dragonwing IQ-9075) + 一個硬體家族。

SoC = Qualcomm QCS9075 · Hexagon NPU + GPU 。同一套 SoC ，落在三種板型上——價值在它上面那層平台。

EXMP-Q911

Module

COM-HPC Mini Module 。

EXEC-Q911

EVK

EVK 開發板 。

APEX-A100

System

Edge AI System 。

Platform · 你不必自己造的那層

四層，平台都幫你備好。

BSP

Yocto 出廠預燒可直接用；Ubuntu 需另燒（取得完整 IO）。

meta-iQ

容器化交付

應用打包成 Docker image 或 IPK 套件。

image / IPK

iqs-launcher

檢查平台 BSP → 線上 pull / 離線 load 對應套件。

一行起 demo

現成 demo

VLM / 多路串流 / 物件偵測，開箱即跑。

3 個

Capabilities · 三種能力面

所見及所得。

VLM · 視覺語言

板上即時跑 LLaVA-1.5-7B，看懂 UVC 攝影機畫面並描述。

11 tokens / sec

多路串流

Streampipe 通用多路推論管線，同時處理多路 1080p / 30fps。

最多 32 路 · INT8

物件偵測

YOLOv10n INT8，在 NPU / GPU 上推論，展示 Hexagon 加速。NPU + GPU · QNN

上半場由於場地限制，不會在燒錄後執行，重頭戲放在下半場。

Why it holds · 同架構、可重現

3.7×

9 路 1080p 同時推論 · AI 端到端吞吐是 NVIDIA Jetson 的 3.7 倍 · 28.4 vs 7.6 FPS / 通道

9 路下仍守住近即時 **30 FPS** · 對手掉到 7.6——差距不在規格峰值，在真實 AI 負載下還撐不撐得住
(同一套軟體架構、同一條 pipeline) 。

28.4

Innodisk · 9 路端到端 (FPS/通道)

7.6

同架構 · NVIDIA Jetson

30W

功耗 · 同場景 Jetson 35W

Flash · 韌體燒錄 (前置)

● 動手

先把工具與接線就位。

工具 (課前)

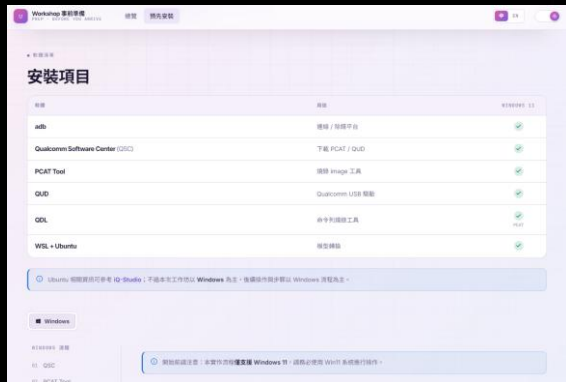
adb · QSC · PCAT · QUD · Docker (Win11 / Ubuntu22.04) 。

pre-install

接線

jumper→EDL ; boot DIP 全 ON (UFS) ; USB-C→Flash/ADB port ; 60W ; DP / 鍵鼠 / 乙太 / UART 。

EDL mode

完整工具安裝步驟見站點 [pre-install.html](#) ; 燒錄完記得把 jumper 切回 Normal 。

Flash · 韌體燒錄 (步驟)

[● 動手](#)

依你的 host , 二選一。

Windows · PCAT

開 QSC → 啟 xPCATApp → 左上選 Q911 → download image → 選 UFS firmware → flash。

[→ 切回 Normal](#)

WSL / Ubuntu · QDL

Ubuntu 端改用 QDL 燒錄。

[QDL](#)

驗收 切回 Normal 、開機進入 OS。

完整步驟與 image 檔以站點 [flashing.html](#) / 公司內部 image 為準 (本頁為摘要)。

Topology · 動手前先講清楚

主機燒錄 + 連線，板子跑 demo。

主機 Win+WSL2+Docker

① 燒錄 (PCAT/QDL) ② 經 ADB (adb shell) 連入板子 ③ WSL2/Docker
主要是下午 iQ-Foundry 的環境。

adb

板子 EXEC-Q911

跑 iQ-Studio demo ; iqs-launcher 在板子端執行。

NPU

demo 的 Docker image 由板子端 pull / load (第一次 ~3 min)。

Init · 環境初始化 (板子端)

在板子上裝好 iQ-Studio 。

```
$ adb shell # 從主機連入板子 ( 或直接在板子終端機 )  
$ git clone https://github.com/InnoIPA/iQ-Studio.git  
$ cd iQ-Studio  
$ ./install.sh
```

install.sh 會備好跑 demo 需要的環境

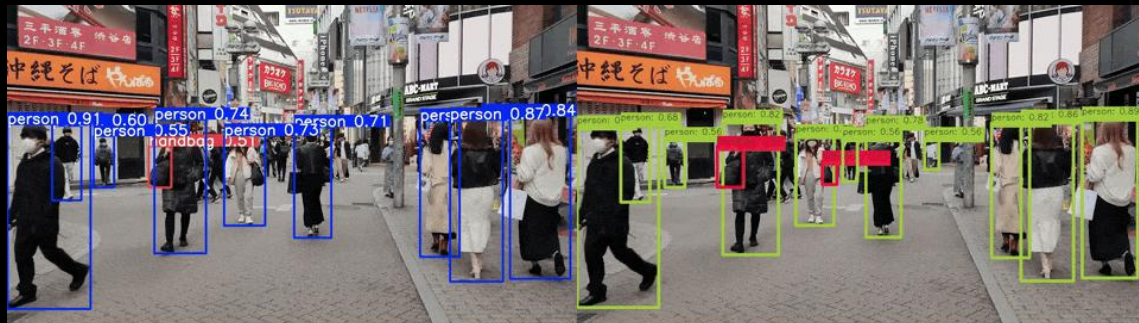
Run · 第一個 demo

一行指令，起一個 AI 應用。

```
$ iqs-launcher --autotag iqs-yolov10n # 物件偵測
```

流程：檢查平台 BSP → 線上 pull / 離線 load 對應 Docker image 或 IPK (第一次 ~3 min) → 在 NPU 上推論，結果輸出成影片。

驗收① 到 output 資料夾開啟結果影片，看到 YOLOv10n 偵測結果。



NVIDIA Jetson AGX ORIN

Qualcomm EXMP-Q911

Run · launcher 全貌與離線

三種用法，加上離線備援。

```
$ iqs-launcher --autotag <app>[:<ver>] # Docker image
$ iqs-launcher --ipk <package> # IPK 套件
$ iqs-launcher --other <command> # 轉旗標給應用
$ iqs-launcher --autotag iqs-streampipe # 多路串流
$ iqs-launcher --autotag iqs-ogenie # VLM 前置
$ iqs-launcher --autotag iqs-vlm-demo # VLM (十分鐘內)
```

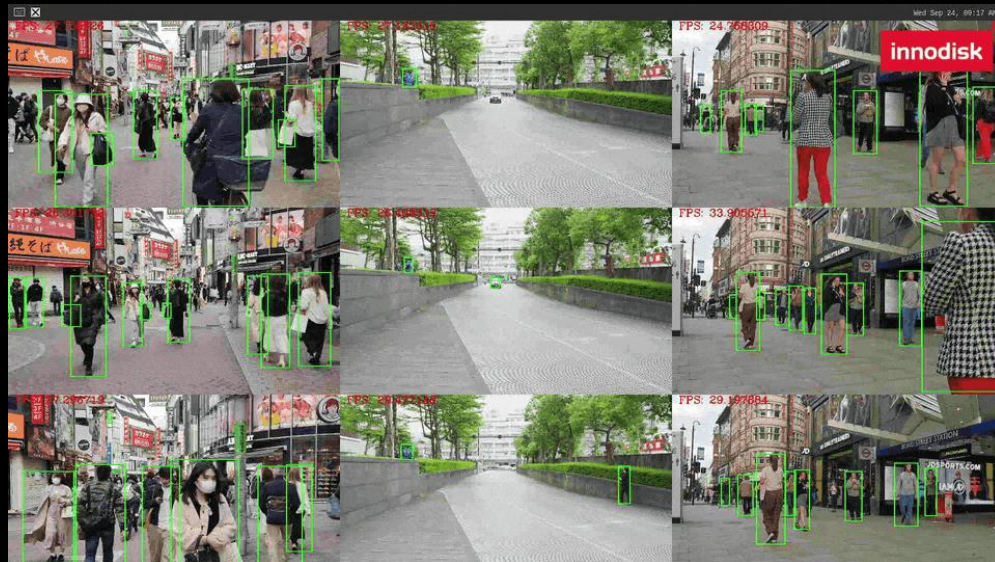


驗收② VLM 兩道指令後，板上看懂攝影機畫面。

離線：.tar 放 binaries/docker-images/、.ipk 放 binaries/ipk/，再跑同指令。現場多人同時線上 pull 會塞，建議走離線暫存。

一份設定，多路同時跑。

Streampipe config → CH: 9 · USB Camera · PPE APP



平台把多路串流 + 模型 + 後處理接好——一份 config 就跑起多路場景。

Bridge · 橋接下午

你剛看到的是**我們的**模型。
下半場，跑**你自己的**。

用 iQ-Foundry 把你自己的 YOLO 26 量化轉換，送進同一條 Streampipe——這就是 Foundry bridge。

iQ-Studio

看到 → 擁有 **iQ-Foundry** →

Flash → Init → Run · 下半場：Bring Your Own Model



innodisk

ARCHITECT INTELLIGENCE